

Software Requirements Specification

for

**FPTU Club Management System
(FCMS)**

Release 1.0

Version 1.0 approved

Prepared by FCMS Development Team

FPT University Club Operations

July 28, 2026

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Document Conventions	3
1.3 Project Scope	3
1.4 References	3
2. Overall Description	4
2.1 Product Perspective	4
2.2 User Classes and Characteristics	4
2.3 Operating Environment	5
2.4 Design and Implementation Constraints	5
2.5 Assumptions and Dependencies	5
3. System Features	6
3.1 Use Case Diagram	6
3.2 Use Case Specifications	6
4. Data Requirements	10
4.1 Logical Data Model	10
4.2 Data Dictionary	11
4.3 Business Rules	12
5. External Interface Requirements	13
5.1 User Interfaces	13
5.2 Software Interfaces	14
5.3 Hardware Interfaces	14
5.4 Communications Interfaces	14
6. Quality Attributes	15
6.1 Usability	15
6.2 Performance	15
6.3 Security	15
6.4 Safety and Data Integrity	15
6.5 Availability Requirements	16
6.6 Robustness and Maintainability	16
6.7 Privacy and Auditability	16
7. Implementation Observations and Acceptance Gaps	17
Appendix A. Principal Lifecycle Flows	18
Appendix B. Role and Capability Matrix	21
Appendix C. Scheduler Catalogue	22
Appendix D. Implementation Traceability	23
Appendix E. Deployment Configuration Handoff	24

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, data, interface, and quality requirements of the FPTU Club Management System (FCMS). It is intended for the development team, quality assurance team, product owner, IC-PDP staff, system administrators, club leaders, and deployment/maintenance personnel. The specification is derived from the current React frontend, Spring Boot backend, Flyway database migrations, security configuration, scheduled jobs, and integration code in the project repository.

The document describes the implemented business domains and the intended production behavior. Where the current implementation and project documentation differ, Section 7 records the observed gap so that acceptance criteria remain explicit.

1.2 Document Conventions

- The keyword **shall** denotes a mandatory requirement; **should** denotes a recommended behavior; **may** denotes an optional behavior.
- Requirement identifiers use domain prefixes such as AUTH, CLUB, REC, EVT, REG, PAY, ATT, FDB, RPT, CON, SEM, KPI, COM, KNO, and OPS.
- Dates use ISO 8601 where exchanged by API; user-facing time is interpreted in the configured application time zone. Currency is VND unless stated otherwise.
- Role names are Guest, Member/Student, Alumni, Club Leader, Vice Leader, IC-PDP, and System Admin.
- Controller → Service → Repository denotes the principal backend execution path. Cross-cutting validation, security filters, events, notifications, audit records, and schedulers may also participate.

1.3 Project Scope

FCMS is a web platform for governing university clubs across semesters. It supports identity and role-based access, club establishment and governance, membership and recruitment, event approval and delivery, paid and free registration, guest OTP verification, attendance, feedback, reports, contribution scoring, appeals, rankings, competitions, notifications, knowledge archives, AI-assisted retrieval, administrative configuration, auditability, and scheduled lifecycle automation.

The system does not process bank cards. Paid registrations use a bank transfer flow reconciled through SePay webhook data or authorized manual verification. The frontend may expose public receiving-bank metadata for QR display, but the SePay webhook API key remains backend-only.

1.4 References

1. Project repository README.md and BACKEND_FLOWS_EVENT_LOGIN_CLOUDINARY.md.
2. Backend source under backend/src/main/java/com/fptu/fcms and Flyway migrations under backend/src/main/resources/db/migration.
3. Frontend routing and feature implementation under frontend/src.

4. Spring Boot, Spring Security, Spring Data JPA, and Flyway documentation applicable to the versions declared in backend/pom.xml.
5. React, Vite, React Router, and Axios documentation applicable to the versions declared in frontend/package.json.
6. SePay webhook, Cloudinary media storage, Google OAuth2, SMTP email, ClamAV scanning, and Gemini API integration contracts configured by the application.

2. Overall Description

2.1 Product Perspective

FCMS is a browser-based, multi-role information system. The React client calls JSON REST endpoints exposed by a Spring Boot application. Requests pass through CORS and security filters, JWT authentication, controller-level authorization, transactional domain services, Spring Data repositories, and SQL Server. Domain events and scheduled jobs perform asynchronous or time-driven lifecycle work. External services provide OAuth2 login, email/OTP delivery, file storage, malware scanning, bank transaction notification, and AI retrieval.

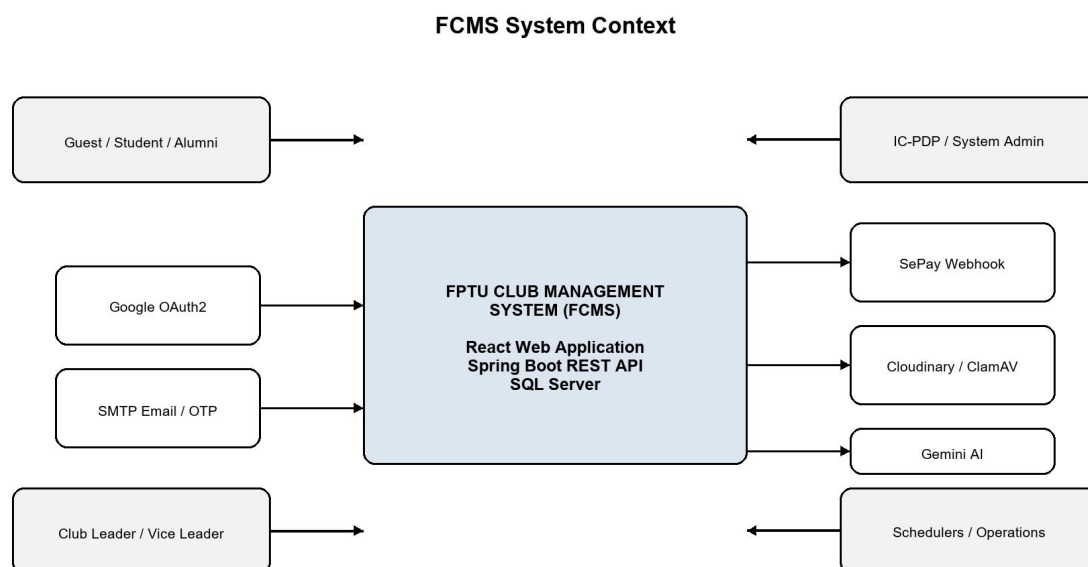


Figure 1. Context diagram for FPTU Club Management System

2.2 User Classes and Characteristics

User class	Characteristics	Primary capabilities
Guest	Unauthenticated visitor or external event participant	Browse public clubs/events; register for eligible events; verify OTP; recover guest registration; submit token-based feedback.
Member / Student	Authenticated FPT student and club participant	Manage profile; apply to clubs; register/pay for events; access tickets; attend; provide feedback; view contributions; submit appeals.
Alumni	Authenticated alumnus-facing frontend role	Browse alumni pages, clubs, events, and network features; backend-specific alumni rules are not yet isolated.

User class	Characteristics	Primary capabilities
Club Leader	Primary officer responsible for a club	Manage club information, board, members, recruitment, events, registrations, attendance, reports, contributions, blacklist, posts, and knowledge.
Vice Leader	Delegated club officer	Performs most operational leader functions subject to assignment and permission checks.
IC-PDP	Institutional club-governance staff	Review events/reports; manage clubs, reassignment, discipline, KPI, competition, recruitment oversight, attendance correction, and knowledge.
System Admin	Platform administrator	Manage users, semesters, system configuration, knowledge archive, security and operational settings.
External service	Trusted system-to-system actor	Google OAuth callback, SePay webhook, SMTP delivery, Cloudinary, ClamAV, and Gemini interactions.

2.3 Operating Environment

- Client: modern desktop or mobile web browser running the Vite-built React application.
- Backend: Java runtime and Spring Boot application as declared by backend/pom.xml; deployable as a web service/container, including Azure App Service.
- Database: Microsoft SQL Server with schema evolution managed by Flyway migrations.
- Frontend hosting: static hosting such as Vercel with environment variables prefixed VITE_ compiled into the client bundle.
- Network: HTTPS between browser and backend; outbound HTTPS to OAuth2, SePay, Cloudinary, and Gemini; SMTP to the configured mail provider.
- Optional infrastructure: Redis-backed or equivalent rate limiting/cache where configured; reachable ClamAV daemon for document scanning.

2.4 Design and Implementation Constraints

ID	Constraint
CO-01	Role-based authorization shall be enforced in the backend; hiding a frontend route is not sufficient access control.
CO-02	Secrets such as JWT signing material, OAuth credentials, SMTP credentials, Cloudinary secret, database password, and SEPAY_WEBHOOK_API_KEY shall remain server-side environment configuration.
CO-03	VITE_ variables are public build-time values. They may contain display bank metadata but shall not contain webhook/API secrets.
CO-04	Database changes shall be forward-only Flyway migrations and shall preserve referential integrity.
CO-05	Uploaded documents shall satisfy extension, MIME/signature, size, and malware checks appropriate to their domain.
CO-06	Payment webhooks and QR attendance operations shall be idempotent because providers and clients can retry.
CO-07	Time-sensitive rules shall use server time and configurable thresholds where SystemConfig provides them.
CO-08	Personal data and operational audit data shall be accessible only to authorized roles and purposes.

2.5 Assumptions and Dependencies

- FPT identity domains and allowed-email records are maintained correctly for account eligibility.
- Each production environment supplies valid SQL Server connectivity, JWT configuration, frontend/backend URLs, and CORS origins.

- Google OAuth2 redirect URIs match the deployed frontend/backend hostnames.
- SePay is configured to call the public backend webhook endpoint with the same API key stored in the backend environment.
- Azure and Vercel bank name, account number, account name, and branch metadata are identical; only public display values use VITE_ prefixes.
- SMTP, Cloudinary, ClamAV, and Gemini services are available when their dependent features are enabled; failures are surfaced without corrupting core transactions.
- A semester is configured before semester-bound club, recruitment, event, KPI, ranking, or competition operations are executed.

3. System Features

3.1 Use Case Diagram

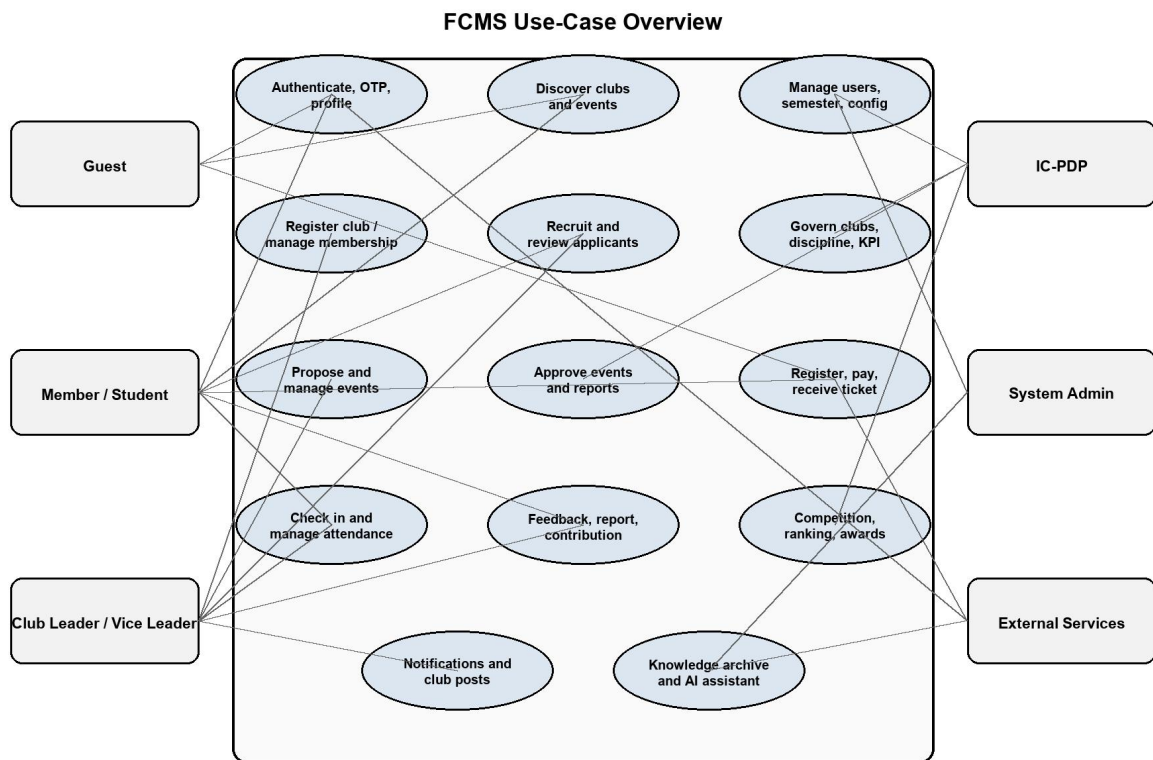


Figure 2. Use case overview for FPTU Club Management System

3.2 Use Case Specifications

3.2.1 UC-01 – Authenticate, activate, and recover an account

Field	Specification
Use case	Authenticate, activate, and recover an account
Primary actors	Guest; Member; Alumni; Staff
Trigger	User opens login/register/recovery or returns from Google OAuth2.
Preconditions	The account is eligible under FPT domain/allowed-email policy; for protected actions the account is active.

Field	Specification
Main flow	1) Validate email and request data. 2) Create or locate the account. 3) Send and verify OTP when required. 4) Hash local passwords with BCrypt or accept a validated OAuth2 identity. 5) Issue JWT. 6) Frontend stores the authenticated session and routes by role.
Alternatives / exceptions	Expired/invalid OTP, locked or inactive account, unauthorized email domain, duplicate identity, invalid credentials, OAuth2 failure, or rate limit returns a controlled error.
Postconditions	A valid session is established or the recovery/activation state is updated; the action is auditable.
Implementation trace	AuthController → AuthServiceImpl / OTPServiceImpl → UserAccount repositories; JwtAuthenticationFilter and OAuth2SuccessHandler enforce token/OAuth flows.

3.2.2 UC-02 – Register and govern a club

Field	Specification
Use case	Register and govern a club
Primary actors	Member; Club Leader; Vice Leader; IC-PDP
Trigger	Eligible student creates a founding-club request or an officer maintains an existing club.
Preconditions	Student has no conflicting leader role or active discipline and satisfies active-club constraints.
Main flow	1) Enter club information. 2) Select founding board. 3) Validate at least five founders with exactly one Leader, one Vice Leader, and at least three Members. 4) Persist registration and club/member records according to implemented workflow. 5) Maintain board, club profile, members, blacklist, discipline, and personnel reassignment under authorization.
Alternatives / exceptions	Duplicate club, invalid founder composition, ineligible founder, membership limit, or unauthorized modification is rejected. Current submit behavior auto-creates/approves the club; the separate review flow remains a recorded implementation gap.
Postconditions	Club governance state, memberships, board roles, and audit records are consistent.
Implementation trace	ClubRegistrationController / ClubController / ClubManagementController / ClubMemberController → ClubRegistrationServiceImpl / ClubServiceImpl / ClubBoardServiceImpl.

3.2.3 UC-03 – Run recruitment and select applicants

Field	Specification
Use case	Run recruitment and select applicants
Primary actors	Club Leader; Vice Leader; Member; IC-PDP
Trigger	Officer opens a recruitment cycle and a member submits an application.
Preconditions	Cycle and semester are active; applicant is eligible, not blacklisted, and has no duplicate active application.
Main flow	1) Create/open cycle. 2) Publish application form and deadlines. 3) Applicant submits details and optional CV. 4) Reviewers screen applications and schedule interviews. 5) Record decision. 6) Passing applicant receives a membership. 7) Send reminders and notifications.
Alternatives / exceptions	CV over 10 MB, more than five withdrawals, withdrawal cooldown under three hours, or active-club plus pending-application total reaching three is rejected by current service rules.
Postconditions	Application, review, interview, withdrawal history, and resulting membership reflect the final decision.
Implementation trace	RecruitmentCycleController / RecruitmentApplicationController / RecruitmentReviewController / WithdrawLogController → corresponding ServiceImpl classes.

3.2.4 UC-04 – Propose, approve, and operate an event

Field	Specification
Use case	Propose, approve, and operate an event
Primary actors	Club Leader; Vice Leader; IC-PDP
Trigger	Authorized officer creates or resubmits an event proposal.
Preconditions	Club and semester are eligible; creator has event permission; required event data and timing are valid.

Field	Specification
Main flow	1) Save DRAFT. 2) Validate new-event lead time of at least 14 days or resubmission lead time of at least 7 days. 3) Submit proposal. 4) IC-PDP checks schedule conflicts and approves/rejects. 5) Configure registration policy and assignments. 6) Scheduler advances lifecycle. 7) Organizer may cancel with a reason under policy. 8) Complete delivery and start reporting.
Alternatives / exceptions	Validation error, exceeded configurable attempts, active cooldown, schedule conflict, invalid state transition, or missing assignment prevents progression.
Postconditions	Event state, approval evidence, assignments, capacity, notifications, and audit trail are synchronized.
Implementation trace	EventController / ICPDPEventController → EventServiceImpl / EventProposalValidatorImpl / EventStateMachineServiceImpl / EventPermissionServiceImpl.

3.2.5 UC-05 – Register, pay, cancel, refund, and receive a ticket

Field	Specification
Use case	Register, pay, cancel, refund, and receive a ticket
Primary actors	Member; Guest; Club operator; SePay
Trigger	Participant registers for an event with an open registration window.
Preconditions	Event is eligible; capacity/policy admits the participant; participant is not already registered.
Main flow	1) Validate audience, window, attempts, cooldown, quota, and ticket count. 2) Confirm a free registration immediately or hold paid quota for 30 minutes. 3) Display bank/QR information. 4) Receive and authenticate SePay webhook. 5) Match transfer content/amount idempotently. 6) Confirm registration and issue ticket. 7) On cancellation, calculate refund tier and record outcome.
Alternatives / exceptions	Maximum two attempts with a 30-minute cooldown; paid group purchase supports one to four tickets; unmatched/late payment remains pending or expires; manual verification requires authorization.
Postconditions	A confirmed registration has a unique ticket; capacity is correct; payment and refund records are auditable.
Implementation trace	EventRegistrationApiController / GuestRegistrationController / SePayWebhookController → EventRegistrationServiceImpl / GuestRegistrationServiceImpl / SePayWebhookService.

3.2.6 UC-06 – Verify a guest and support walk-in admission

Field	Specification
Use case	Verify a guest and support walk-in admission
Primary actors	Guest; Club operator
Trigger	External participant starts guest registration or arrives at event.
Preconditions	Event accepts the applicable participant type and registration channel.
Main flow	1) Capture guest identity/contact. 2) Send OTP. 3) Verify OTP within validity and attempt limits. 4) Create or recover registration by reference. 5) For walk-in, authorized operator validates capacity and records admission. 6) Send payment/ticket email where applicable.
Alternatives / exceptions	Expired OTP, attempt limit, duplicate contact, closed registration, insufficient capacity, or ineligible event blocks registration.
Postconditions	Guest registration is verified, traceable, and eligible for downstream ticket, attendance, and token feedback.
Implementation trace	GuestRegistrationController / WalkInController → GuestRegistrationServiceImpl / GuestOtpAttemptService / WalkInServiceImpl.

3.2.7 UC-07 – Record attendance, collect feedback, and approve an event report

Field	Specification
Use case	Record attendance, collect feedback, and approve an event report
Primary actors	Organizer; Member; Guest; IC-PDP
Trigger	Organizer opens attendance and later submits the event report.

Field	Specification
Preconditions	Event has one valid active attendance session; participant registration is confirmed; reporting waits until event/feedback prerequisites are satisfied.
Main flow	1) Open attendance session. 2) Scan QR or record authorized correction. 3) Idempotently write attendance. 4) Finalize session and mark missing confirmed participants ABSENT. 5) Invite eligible PRESENT participants to feedback. 6) Close feedback window. 7) Upload and scan report PDF. 8) Snapshot statistics/evidence. 9) IC-PDP approves or rejects report.
Alternatives / exceptions	Duplicate scans are safe; invalid/expired token, unauthorized correction, open feedback, unclosed attendance, pending payment, invalid PDF, malware result, or invalid event state blocks the relevant step.
Postconditions	Attendance, feedback summary, report review, and evidence snapshots are immutable enough for contribution calculation and audit.
Implementation trace	AttendanceController / AttendanceSessionController / FeedbackController / ReportController → AttendanceServiceImpl / FeedbackServiceImpl / ReportUploadServiceImpl.

3.2.8 UC-08 – Calculate contribution, resolve appeals, and publish ranking

Field	Specification
Use case	Calculate contribution, resolve appeals, and publish ranking
Primary actors	Club officer; Member; IC-PDP
Trigger	An event report is approved and contribution allocation begins.
Preconditions	Attendance/report evidence exists and the contribution batch is in an allowed state.
Main flow	1) Create contribution batch. 2) Calculate base 100 plus bonus minus penalty. 3) Apply role weights: core 50, support 30, participant 20. 4) Assign grade A ≥ 140, B ≥ 120, C ≥ 90, else D. 5) Publish member values. 6) Accept appeals for 24 hours. 7) Ensure reviewer is not self-finalizing. 8) Finalize and snapshot rankings.
Alternatives / exceptions	Missing evidence, duplicate batch, appeal outside window, self-review, or invalid batch state is rejected.
Postconditions	Final contribution records, appeal decisions, and ranking snapshots are consistent and auditable.
Implementation trace	ContributionBatchController / MemberRankingController → ContributionBatchServiceImpl / ContributionServiceImpl / MemberRankingServiceImpl.

3.2.9 UC-09 – Manage semesters, KPI evaluation, competition, and awards

Field	Specification
Use case	Manage semesters, KPI evaluation, competition, and awards
Primary actors	System Admin; IC-PDP
Trigger	Administrator configures a semester or IC-PDP evaluates clubs.
Preconditions	Actor has administrative authority and referenced semester/club data exists.
Main flow	1) Create semester without overlap and maintain one active semester. 2) Evaluate closure blockers or force-close with reason and audit. 3) Calculate club KPI using completion, membership, attendance, report, contribution, and compliance indicators. 4) Assign Continue/Improvement/Warning/Suspend decision. 5) Configure competition and score activity, feedback, participation, compliance, and engagement. 6) Publish rankings/awards and notify top clubs.
Alternatives / exceptions	Overlap, multiple active semesters, unresolved close blockers, invalid scores, or unauthorized force-close is rejected. Competition engagement is currently a placeholder metric.
Postconditions	Semester, KPI, evaluation, competition score, award, and snapshot data are preserved.
Implementation trace	AdminSemesterController / SemesterController / ClubDashboardController / CompetitionController → SemesterServiceImpl / ClubDashboardServiceImpl / CompetitionServiceImpl.

3.2.10 UC-10 – Manage knowledge archives and ask the AI assistant

Field	Specification
Use case	Manage knowledge archives and ask the AI assistant

Field	Specification
Primary actors	Club officer; IC-PDP; Admin; authorized member
Trigger	Authorized user uploads knowledge or asks a scoped question.
Preconditions	Archive scope grants visibility; file is MD/TXT/PDF up to 5 MB and passes validation/scanning.
Main flow	1) Upload metadata and file. 2) Mark archive pending. 3) Publish asynchronous ingestion event. 4) Extract and split text into chunks of 500 with overlap 100. 5) Store searchable chunks. 6) Retrieve the top three visible chunks for a query. 7) Call Gemini with grounded context. 8) Return fallback when AI is unavailable and write AI audit log.
Alternatives / exceptions	Unauthorized scope, unsupported content, oversize file, malware, extraction failure, or AI/provider failure returns a controlled status without exposing restricted content.
Postconditions	Knowledge status, chunks, response, citations/context identifiers, and AI audit are recorded as applicable.
Implementation trace	KnowledgeArchiveController / AIChatController → KnowledgeArchiveServiceImpl / KnowledgeIngestionServiceImpl / AIChatServiceImpl.

4. Data Requirements

4.1 Logical Data Model

4.1.1 Conceptual Model

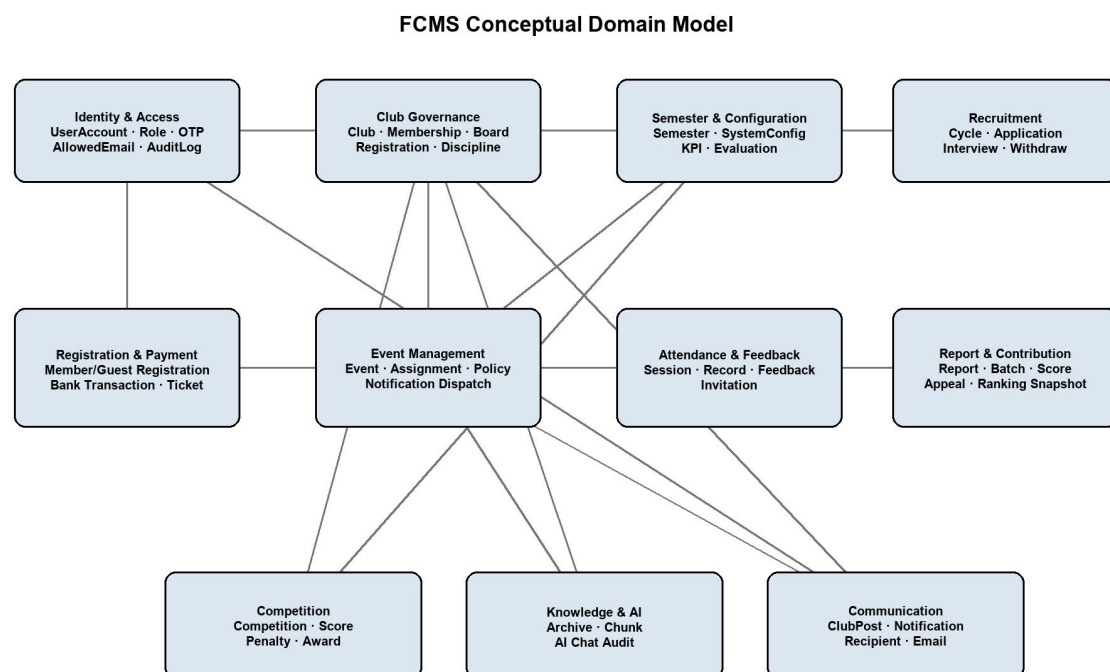


Figure 3. Conceptual domain model for FPTU Club Management System

4.1.2 Logical Data Organization

The persistence model contains 62 JPA entity classes. Entities are grouped by lifecycle ownership and connected through explicit identifiers and relationships. SQL Server constraints and Flyway migrations are the authoritative physical schema; this SRS presents the logical aggregates required for behavior and test traceability.

4.2 Data Dictionary

Entity / group	Domain	Key data	Principal relationships
UserAccount	Identity	user id, email, password/OAuth identity, profile, status	SystemRole; memberships; registrations; audit
SystemRole / ClubRole / EventRole	Authorization	role code, name, scope	Users, memberships, event assignments
AllowedEmail	Identity policy	email/domain, active state	Registration eligibility
OTPVerification / GuestVerificationOtp	Verification	subject, code hash/value, expiry, attempts, status	User or guest flow
AuditLog / SchedulerLog / AIChatAuditLog	Audit	actor/job, action, target, timestamp, outcome, metadata	Cross-domain evidence
Club	Club	club id, name, abbreviation, description, status, semester	Memberships, events, posts, KPI
ClubRegistration	Club formation	proposal, founder, status, review data	Founding members
ClubRegistrationMember	Club formation	registration, user, proposed role	ClubRegistration; UserAccount
ClubMembership	Membership	club, user, role, joined/left dates, status	UserAccount; Club
ClubBlacklist	Discipline	club, user, reason, active period	Recruitment eligibility
DisciplineLog	Discipline	subject, violation, action, evidence, dates	User/club governance
PersonnelReassignLog	Governance	club, old/new assignee, reason, decision	Leadership continuity
ClubPost	Content	club, author, title/body, publication status	Public/member club space
RecruitmentCycle	Recruitment	club, semester, open/close dates, status	Applications, reminders
RecruitmentApplication	Recruitment	cycle, applicant, answers, CV, status	Review/interview/withdraw
InterviewerAssignment / InterviewSchedule	Recruitment	reviewer, applicant, schedule, result	RecruitmentApplication
WithdrawLog / RecruitmentReminder	Recruitment	application/cycle, time, reason or reminder type	Application lifecycle
Semester	Academic period	code/name, start/end, status, closure metadata	Clubs, events, ranking, competition
SystemConfig	Configuration	key, typed value, description, active/version	Policy services and schedulers
Event	Event	club, semester, schedule, venue, audience, capacity, fee, status	Assignments, registrations, attendance, report
EventAssignment	Event staffing	event, user, event role, assignment state	Permissions and contribution
EventRegistrationPolicy	Registration policy	event, window, limits, guest/group/payment flags	Registration services
EventRegistration	Member registration	event, user, status, channel, ticket, amount	Payment and attendance
GuestEventRegistration	Guest registration	event, guest identity/contact, reference, status, ticket	OTP, payment, attendance
BankPaymentTransaction	Payment	provider id, transfer reference/content, amount, status, matched registration	SePay reconciliation/refund evidence
AttendanceSession	Attendance	event, open/close time, token/config, status	Attendance records
AttendanceRecord	Attendance	session, participant, status, method, timestamps, correction	Registration and feedback eligibility
EventFeedback / EventFeedbackInvitation	Feedback	event/participant, rating, comment, token, status	Report statistics
EventReport	Reporting	event, file, statistics snapshot, evidence, review status	Contribution batch
EventReportReminderLog	Reporting	event/report, reminder type/time/outcome	Scheduler
EventContribution	Contribution	event, member, role/base/bonus/penalty, score, grade	Batch and rankings
ContributionBatch	Contribution	event, status, open/final dates, totals	Contributions and appeals
ContributionAppeal	Appeal	contribution, requester, reason/evidence, decision, reviewer	Final contribution
MemberPerformance / MemberRankingSnapshot	Ranking	member, semester, metrics, rank, snapshot time	Leaderboards
ClubKPI / ClubEvaluation	Governance	club/semester, metric values, weighted score, decision	Dashboard
Competition	Competition	semester, rules, period, status	Scores, penalties, awards
CompetitionScore / CompetitionPenalty / CompetitionAward	Competition	club, metric/penalty/award, points, evidence	Competition ranking
Notification / NotificationRecipient	Communication	type, title/body, target, read state, delivery metadata	Users and domain events
EventNotificationDispatch	Communication	event, notification type, recipients, dispatch state	Idempotent notification
KnowledgeArchive	Knowledge	scope, owner, file, type, status, visibility	Chunks and AI retrieval
KnowledgeChunk	Knowledge	archive, sequence, text, embedding-store id	Search/RAG

4.3 Business Rules

Rule	Requirement
AUTH-01	Only emails permitted by the FPT domain or AllowedEmail policy may register/authenticate where the restriction applies.
AUTH-02	A pending/inactive account shall not receive normal protected access until activation requirements are satisfied.
AUTH-03	Local passwords shall be BCrypt-hashed; plaintext passwords shall never be stored or logged.
AUTH-04	JWT identity and backend authorization are authoritative for protected API access.
AUTH-05	OTP verification shall enforce expiry, attempt limits, one-time use, and applicable rate limits.
AUTH-06	OAuth2 identities shall be mapped to an eligible internal account before JWT issuance.
CLUB-01	A founding club requires at least five students: exactly one Leader, exactly one Vice Leader, and at least three Members.
CLUB-02	A founder shall be a student and shall satisfy the configured/implemented active-club membership limit.
CLUB-03	A proposed leader shall not have an active discipline and shall not simultaneously lead another active club.
CLUB-04	Club names/abbreviations that must be unique shall be rejected when duplicated.
CLUB-05	Only authorized club officers or institutional staff may modify club governance data.
CLUB-06	Blacklist and discipline decisions shall retain reason, time range, actor, and evidence where provided.
CLUB-07	Leadership reassignment shall preserve old/new assignee and decision history.
REC-01	Applications are accepted only while the recruitment cycle and associated semester are active/open.
REC-02	An applicant shall not be blacklisted by the target club and shall not have a duplicate active application.
REC-03	The current service prevents applying when active-club memberships plus pending applications reach three.
REC-04	An applicant may withdraw no more than five times and must observe a three-hour withdrawal cooldown.
REC-05	An uploaded recruitment CV shall not exceed 10 MB and shall pass file validation.
REC-06	A passed/final accepted recruitment decision creates the corresponding club membership exactly once.
EVT-01	A new event shall be scheduled at least 14 days after submission; a resubmitted event at least seven days after resubmission.
EVT-02	Only DRAFT or REJECTED events may enter the submit/resubmit path defined by the state machine.
EVT-03	Submission attempts and cooldown are configurable and shall be validated atomically.
EVT-04	IC-PDP approval shall check material schedule/resource conflicts before approval.
EVT-05	Every event status transition shall be allowed by the event state machine; direct invalid transitions are rejected.
EVT-06	Cancellation shall record actor, reason, time, and downstream participant/payment handling.
EVT-07	Assignments shall grant only the event-scoped permissions associated with their event roles.
REG-01	Registration is accepted only within the configured window and for an allowed participant type/channel.
REG-02	Capacity checks and reservation writes shall be transactional to prevent oversubscription.
REG-03	A participant may attempt registration at most twice within the policy window and must observe a 30-minute cooldown after the applicable failure/cancellation.
REG-04	Paid group registration supports one to four tickets and no registration owner may hold more than four tickets in that purchase.
REG-05	A paid registration holds quota for 30 minutes while payment is pending; expiry releases quota.
REG-06	A ticket is issued only after a free registration is confirmed or a paid registration is verified.
REG-07	Participant cancellation shall observe the 24-hour deadline where configured by the implemented policy.
PAY-01	SePay webhook requests shall authenticate with SEPAY_WEBHOOK_API_KEY and be handled idempotently.
PAY-02	Payment matching shall validate the receiving account, transaction identity/content, amount, and registration state.
PAY-03	Manual payment verification shall require an authorized operator and retain audit evidence.
PAY-04	Participant cancellation refund: at least seven days before event = 100%; at least three days = 75%; at least 24 hours = 50%; under 24 hours = 0%.
PAY-05	Organizer cancellation results in a 100% refund obligation for confirmed paid registrations.
PAY-06	Azure backend and Vercel frontend bank account display metadata shall match; SEPAY_WEBHOOK_API_KEY shall never be deployed to Vercel.
GST-01	Guest registration requires contact verification through an unexpired OTP with bounded attempts.

Rule	Requirement
GST-02	Guest registration can be recovered using its reference only after identity/contact verification required by the flow.
GST-03	Walk-in admission requires operator authority, event eligibility, and remaining capacity.
ATT-01	At most one attendance session may be active for an event at a time.
ATT-02	QR/check-in processing shall be idempotent and shall not create duplicate attendance records.
ATT-03	Only confirmed eligible registrations may be checked in unless an authorized correction/walk-in path applies.
ATT-04	Finalizing a session marks confirmed participants without a record as ABSENT.
ATT-05	Attendance corrections shall retain the correcting actor, reason, old value, new value, and timestamp.
FDB-01	Feedback is accepted only after the event and during the configured feedback window.
FDB-02	Member/guest feedback eligibility requires PRESENT attendance and the correct identity or guest token.
FDB-03	Default feedback-quality thresholds are external-present ≥ 10 , responses ≥ 5 , response rate $\geq 20\%$, average rating ≥ 4.0 , and positive rate $\geq 70\%$, unless configuration overrides them.
RPT-01	An event report may be submitted only from COMPLETED or REPORT_REJECTED status.
RPT-02	Report submission requires closed attendance, closed feedback, and no pending payment condition that blocks final statistics.
RPT-03	A report PDF shall be no larger than 10 MB and shall pass extension, MIME/magic-header, and ClamAV checks.
RPT-04	Report statistics and evidence shall be snapshotted at submission/review time.
RPT-05	Approving the report enables the contribution workflow; rejection returns actionable review information.
CON-01	Contribution score equals base 100 plus bonus minus penalty, subject to domain validation.
CON-02	Default role components are core 50, support 30, and participant 20.
CON-03	Grades are A for score ≥ 140 , B for score ≥ 120 , C for score ≥ 90 , and D otherwise.
CON-04	The appeal window is 24 hours from publication/opening as implemented.
CON-05	A reviewer shall not finalize the review of their own appeal.
CON-06	Finalized batches and ranking snapshots shall not silently change when source data changes later.
SEM-01	Semester date ranges shall not overlap and at most one semester may be active.
SEM-02	Normal semester closure shall fail while defined blockers remain.
SEM-03	Force closure requires privileged authority, a non-empty reason, and an audit record.
SEM-04	Semester closure creates/preserves member ranking snapshots and may notify the top three.
KPI-01	Club KPI combines event completion, membership, attendance, report, contribution, and compliance measures using configured weights.
KPI-02	Club evaluation maps the resulting evidence to Continue, Improvement, Warning, or Suspend.
COM-01	Competition score components have maxima: activity 25, feedback 20, participation 15, compliance 15, engagement 25.
COM-02	Competition penalties and awards shall be traceable to the competition, club, actor, score/evidence, and time.
KNO-01	Knowledge uploads support MD, TXT, and PDF up to 5 MB and require security validation.
KNO-02	Archive visibility/scope shall be applied before retrieval; unauthorized chunks shall never enter AI context.
KNO-03	Ingestion splits extracted text into chunks of 500 with overlap 100 according to the current service configuration.
KNO-04	AI retrieval uses the top three visible archive chunks and records an AI audit event.
KNO-05	Gemini/provider failure shall return a safe fallback rather than exposing secrets or unrelated archive data.
OPS-01	Scheduled jobs shall be retry-safe and record sufficient execution outcome to diagnose failures.
OPS-02	Notifications shall avoid duplicate dispatch for the same domain event and recipient where dispatch tracking exists.
OPS-03	State-changing domain operations shall execute transactionally when partial updates would violate consistency.

5. External Interface Requirements

5.1 User Interfaces

ID	User-interface requirement
UI-01	Public pages shall provide landing, club list/detail, event list/detail, login/register/recovery, guest registration/lookup/status, public feedback, and competition results.
UI-02	Protected layouts shall route users by role and reject unauthorized navigation even if a URL is entered directly.

ID	User-interface requirement
UI-03	Member pages shall expose clubs, applications, events, tickets, pending feedback, contributions, appeals, notifications, leaderboard, and profile.
UI-04	Leader/Vice pages shall expose club space, events, assignments, check-in, walk-in, registrations, attendance, feedback, reports, contributions, members, recruitment, blacklist where authorized, club info, and knowledge.
UI-05	IC-PDP pages shall expose club dashboard/management, event approval/management, report review, personnel reassignment, discipline, recruitment, competition, attendance correction, feedback, contribution, knowledge, and profile.
UI-06	Admin pages shall expose semesters, users, system configuration, knowledge archive, feedback oversight, and profile.
UI-07	Payment screens shall show bank ID/name, account number/name, branch, amount, transfer content, QR data, and expiry without exposing webhook credentials.
UI-08	Status, validation, empty, loading, success, and failure states shall be explicit and accessible.

5.2 Software Interfaces

ID	Interface	Contract
SI-01	Frontend ↔ Backend	RESTful JSON over HTTPS; Axios attaches JWT for protected calls; multipart form data for supported uploads.
SI-02	SQL Server	Spring Data JPA persistence with Flyway-managed schema migrations and database constraints.
SI-03	Google OAuth2	Authorization redirect/callback; validated identity maps to an internal user and returns to the configured frontend URL.
SI-04	SePay	Authenticated webhook delivers bank transactions; backend verifies key, de-duplicates provider transaction, and reconciles registration/payment.
SI-05	Cloudinary	Stores approved image/document content; backend retains secure identifier/URL metadata and deletion lifecycle where supported.
SI-06	SMTP	Sends activation, OTP, reminders, ticket/payment, feedback, and operational notifications.
SI-07	ClamAV	Scans security-sensitive uploaded files before acceptance/processing.
SI-08	Gemini AI	Receives a grounded prompt built only from authorized retrieved knowledge chunks; errors use safe fallback behavior.
SI-09	VietQR / QR rendering	Frontend builds public payment/check-in QR representations from approved public data and server-issued tokens.

5.3 Hardware Interfaces

- No specialized server-side hardware is required beyond the deployment platform and network connectivity.
- A participant or operator device camera may scan QR tickets/attendance tokens through browser capabilities.
- Desktop and mobile displays shall support responsive access to public, participant, and operational pages.
- Banking operations occur through the participant's banking application; FCMS does not directly control banking hardware.

5.4 Communications Interfaces

- HTTPS is mandatory for production browser/API, OAuth2 callback, webhook, storage, and AI traffic.
- JWT bearer tokens protect authenticated REST calls; OTP and recovery tokens have limited lifetime and scope.
- SePay webhook authentication uses the backend-only SEPAY_WEBHOOK_API_KEY; repeated delivery must be safe.

- Email is delivered through authenticated SMTP. Sensitive tokens shall not be written to application logs.
- The current implementation persists notifications and supports client retrieval/polling; a WebSocket transport is not established by the present dependency set.

6. Quality Attributes

6.1 Usability

- UX-01: Navigation, terminology, status labels, and actions shall be consistent across role layouts.
- UX-02: Forms shall explain validation failures at the field or action level and preserve safe user input after recoverable errors.
- UX-03: Critical flows such as payment, OTP, check-in, report submission, and appeal shall display deadline/countdown and final status.
- UX-04: Tables and dashboards shall support practical filtering, paging, and empty/loading states for the expected data volume.

6.2 Performance

- PERF-01: Normal list/detail API calls should complete within two seconds at typical campus load, excluding external-provider latency.
- PERF-02: Capacity, payment, and attendance writes shall use bounded database work and indexes appropriate to lookup keys and states.
- PERF-03: Long-running ingestion, notifications, exports, and scheduled batches shall not block unrelated interactive requests.
- PERF-04: AI retrieval shall limit context to the configured top three chunks to bound latency and data exposure.

6.3 Security

- SEC-01: Backend endpoints shall enforce authentication and role/resource authorization independently of the frontend.
- SEC-02: Production secrets shall be environment-managed and rotated; no live secrets shall be committed or compiled into Vite assets.
- SEC-03: OAuth redirects, CORS origins, and cookie/token handling shall use deployed HTTPS origins and least privilege.
- SEC-04: File uploads shall be size/type/signature validated and malware-scanned before downstream use.
- SEC-05: Webhook and OTP endpoints shall use authentication, rate limiting, replay/idempotency controls, and safe errors.
- SEC-06: Logs shall exclude passwords, OTP codes, JWTs, webhook secrets, OAuth secrets, database credentials, and unnecessary personal data.

6.4 Safety and Data Integrity

- INT-01: Transaction boundaries shall prevent oversold capacity, duplicate ticket issuance, partial membership creation, and partial score finalization.

- INT-02: Destructive or force-close operations shall require explicit authorization, reason, and audit evidence.
- INT-03: Immutable snapshots shall preserve reviewed report, contribution, ranking, and evaluation evidence.
- INT-04: Provider retries and repeated QR submissions shall not duplicate material business effects.

6.5 Availability Requirements

- AVL-01: Core browsing and administration shall fail gracefully if a non-core provider such as Gemini is unavailable.
- AVL-02: Scheduled lifecycle jobs shall recover missed work by querying eligible states, not solely by relying on a single in-memory timer.
- AVL-03: Database backups, migration history, environment configuration, and deployment rollback procedures shall be maintained by operations.
- AVL-04: Health and error monitoring shall distinguish application, database, storage, email, antivirus, payment, and AI failures.

6.6 Robustness and Maintainability

- ROB-01: Invalid state transitions and malformed external input shall return controlled errors without corrupting persisted state.
- ROB-02: Controllers shall remain thin; business policy belongs in services/validators/state machines and data access in repositories.
- ROB-03: New schema changes shall use versioned Flyway migrations and automated tests shall cover security and critical state changes.
- ROB-04: Configuration keys shall have documented defaults, validation, ownership, and safe production values.
- ROB-05: Domain notifications and schedulers shall be idempotent or have dispatch/log records that support safe retry.

6.7 Privacy and Auditability

- PRV-01: User and guest identity/contact data shall be visible only to roles with a legitimate operational purpose.
- PRV-02: Security, governance, payment, attendance correction, report review, appeal, and force-close actions shall be attributable to an actor and time.
- PRV-03: Knowledge retrieval and AI audit shall preserve scope decisions without leaking document contents to unauthorized users.
- PRV-04: Retention and deletion procedures shall account for audit/legal requirements and external storage objects.

7. Implementation Observations and Acceptance Gaps

The following items are observations from the current repository. They are recorded to prevent an implementation detail from being mistaken for an approved requirement. Product ownership should decide whether to align the implementation to this SRS or amend the requirement before production acceptance.

ID	Area	Current observation	Required disposition
GAP-01	Club registration	Submission currently auto-approves/creates the club while separate review states/services also exist.	Choose and test one authoritative approval workflow.
GAP-02	Recruitment limit	README material mentions four clubs while the current application service enforces active memberships plus pending applications below three.	Confirm the policy and align code, UI text, tests, and documentation.
GAP-03	Frontend/backend contract	Frontend references endpoints such as /auth/change-password and /auth/check-email plus several club/member/competition/statistics calls that are not evident as matching backend mappings.	Create an API contract inventory and implement or remove unmatched calls.
GAP-04	Competition engagement	The engagement component is a placeholder in the current scoring implementation.	Define evidence and calculation before using the score for formal awards.
GAP-05	Realtime notifications	README material mentions WebSocket behavior, but the current backend dependency/configuration does not establish a WebSocket channel.	Treat database/email/polling as current behavior or add a designed realtime transport.
GAP-06	Alumni domain	Alumni pages/routes exist in the frontend, but no distinct backend alumni aggregate/service is evident.	Define alumni authorization, data source, and supported actions.
GAP-07	Production security	Repository configuration has development-oriented redirect/CORS/secret patterns that must not be carried unchanged to production.	Use environment-managed secrets, exact HTTPS origins, deployed OAuth redirects, and secret rotation.
GAP-08	Payment deployment	Azure/Vercel environment values are not part of repository source and require coordinated handoff.	Document the environment-variable checklist and verify webhook/account values in a staging payment test.

Appendix A. Principal Lifecycle Flows

Event Proposal and Governance Lifecycle

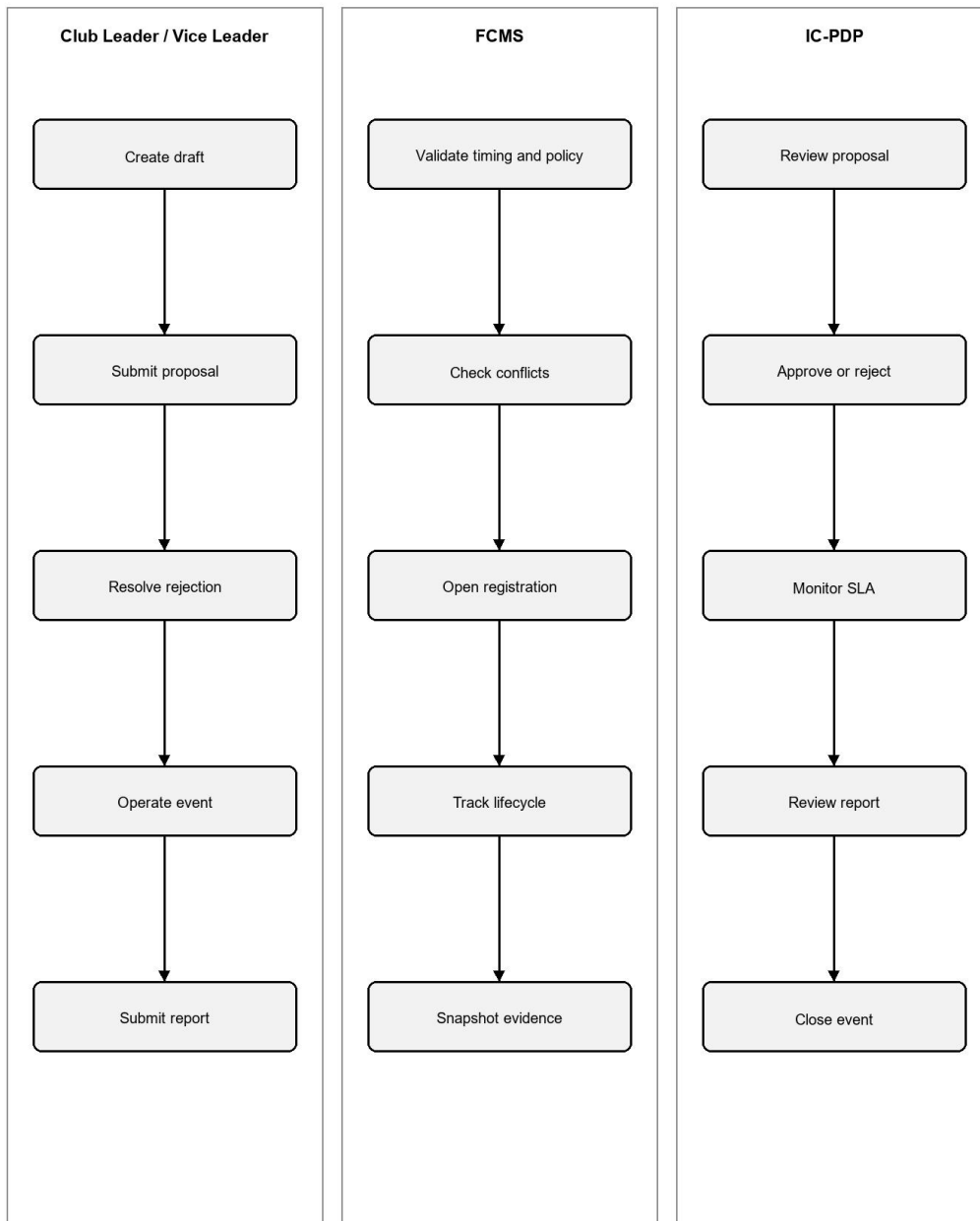


Figure 4. Event proposal, approval, operation, and reporting flow

Registration, SePay Payment and Ticket Flow

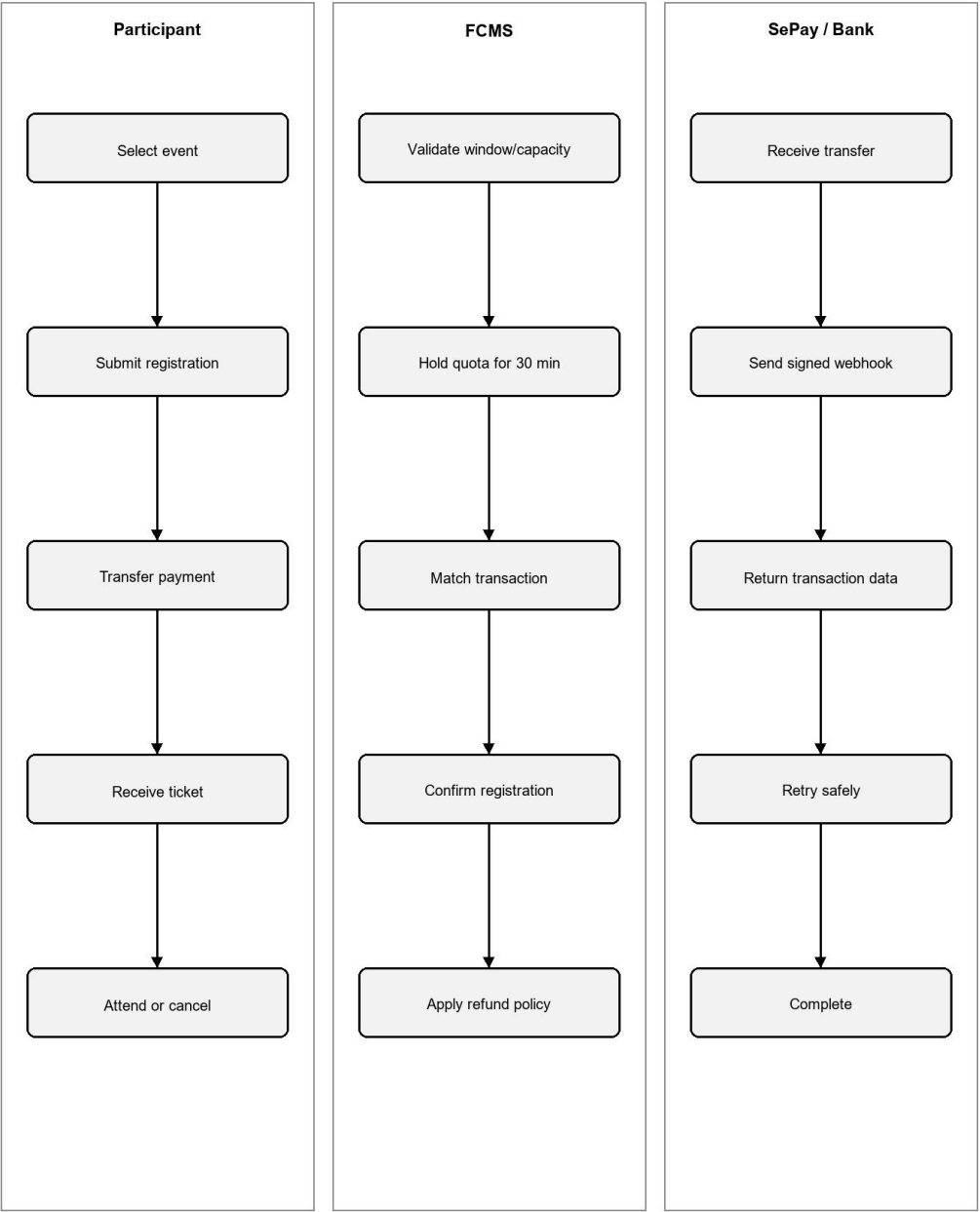


Figure 5. Registration, SePay payment reconciliation, and ticket flow

Attendance-to-Contribution Lifecycle

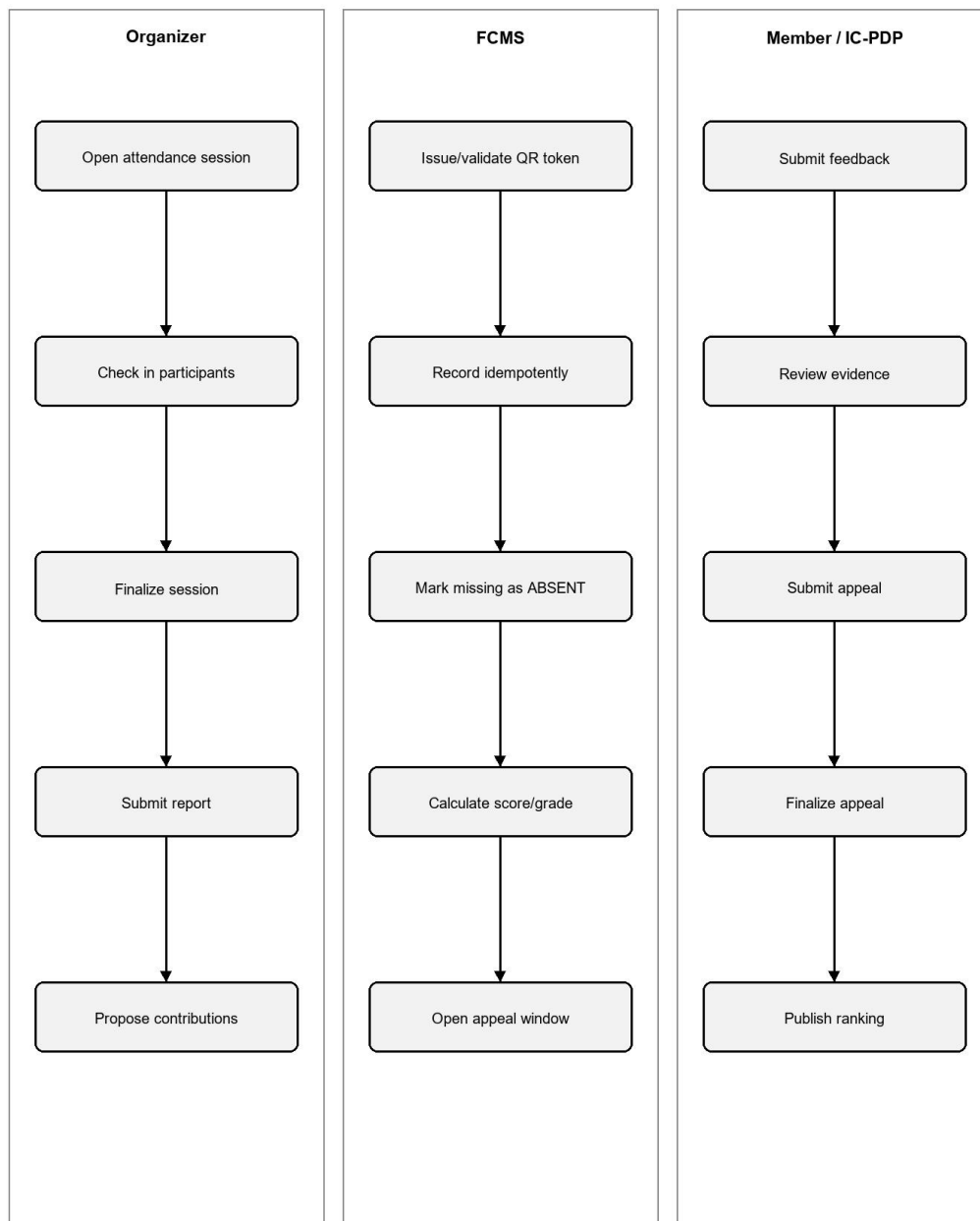


Figure 6. Attendance, feedback, report, contribution, and appeal flow

Appendix B. Role and Capability Matrix

Capability	Guest	Member	Alumni	Leader/Vice	IC-PDP	Admin
Public discovery	Yes	Yes	Yes	Yes	Yes	Yes
Profile and notifications	Limited	Yes	Yes	Yes	Yes	Yes
Apply to club	No	Yes	Policy	Officer context	Oversight	No
Manage members/recruitment	No	No	No	Yes	Oversight	No
Create/manage event	No	Participant	Participant	Yes	Approval/oversight	Config/oversight
Register/pay/ticket	Guest flow	Yes	Yes	Yes	Policy	No
Attendance operations	Participant	Participant	Participant	Yes	Correction/oversight	No
Feedback/report	Token feedback	Feedback	Feedback	Submit report	Review	Oversight
Contribution/appeal	No	View/appeal	View/appeal	Propose/manage	Finalize/oversight	No
Semester/config/users	No	No	No	No	Operational read	Manage
KPI/competition	View public result	View	View	Club view	Manage	Oversight
Knowledge/AI	No	Scoped use	Scoped use	Club manage/use	Manage/use	Manage/use

Appendix C. Scheduler Catalogue

Scheduler class	Responsibility
AppealFinalizeScheduler	Finalize eligible contribution appeals/windows safely.
ClubSizeScheduler	Evaluate club-size conditions and related governance action.
ContributionAppealWindowScheduler	Open/close contribution appeal windows by time/state.
DraftApplicationCleanupScheduler	Clean stale draft recruitment applications.
EventAbsenceScheduler	Apply absence-related event/attendance consequences.
EventLifecycleScheduler	Advance recoverable event lifecycle states.
EventReminderScheduler	Send time-based event reminders.
EventReportReminderScheduler	Remind responsible users about due reports.
EventSlaScheduler	Monitor event workflow service-level deadlines.
FeedbackInvitationScheduler	Issue eligible feedback invitations.
GuestOtpExpiryScheduler	Expire stale guest verification OTP records.
QuotaReleaseScheduler	Release held capacity when reservations no longer qualify.
RecruitmentReminderScheduler	Send recruitment deadline/status reminders.
RegistrationCloseScheduler	Close registration when its configured deadline passes.
SemesterClosureScheduler	Evaluate/perform scheduled semester closure work.
TicketPaymentExpiryScheduler	Expire unpaid ticket holds and release quota.
BaseScheduler	Shared scheduling execution/logging support used by concrete jobs.

Appendix D. Implementation Traceability

Domain	Controllers	Services / logic	Principal entities
Identity and profile	AuthController; ProfileController; AdminUserController	AuthServiceImpl; OTPServiceImpl; UserServiceImpl; AdminUserServiceImpl	UserAccount; SystemRole; AllowedEmail; OTPVerification
Club registration/governance	ClubRegistrationController; ClubController; ClubManagementController; ClubMemberController	ClubRegistrationServiceImpl; ClubServiceImpl; ClubBoardServiceImpl	Club; ClubRegistration; ClubRegistrationMember; ClubMembership
Discipline/reassignment	ClubBlacklistController; DisciplineLogController; ICPDPPersonnelReassignController	ClubBlacklistServiceImpl; DisciplineLogServiceImpl; PersonnelReassignServiceImpl	ClubBlacklist; DisciplineLog; PersonnelReassignLog
Recruitment	RecruitmentCycleController; RecruitmentApplicationController; RecruitmentReviewController; WithdrawLogController	RecruitmentCycleServiceImpl; RecruitmentApplicationServiceImpl; RecruitmentReviewServiceImpl	RecruitmentCycle; RecruitmentApplication; InterviewSchedule; InterviewerAssignment; WithdrawLog
Events and assignments	EventController; ICPDPEventController	EventServiceImpl; EventProposalValidatorImpl; EventStateMachineServiceImpl; EventPermissionServiceImpl; EventAssignmentAccessServiceImpl	Event; EventAssignment; EventRegistrationPolicy
Registration/payment	EventRegistrationApiController; GuestRegistrationController; SePayWebhookController; WalkInController	EventRegistrationServiceImpl; GuestRegistrationServiceImpl; RegistrationAllocationServiceImpl; EventCapacityServiceImpl; SePayWebhookService; WalkInServiceImpl	EventRegistration; GuestEventRegistration; BankPaymentTransaction
Attendance	AttendanceController; AttendanceSessionController	AttendanceServiceImpl; AttendanceSessionServiceImpl; AttendanceTokenServiceImpl	AttendanceSession; AttendanceRecord
Feedback/report/export	FeedbackController; ReportController; EventExportController	FeedbackServiceImpl; FeedbackSummaryServiceImpl; ReportUploadServiceImpl; EventReportStatisticsServiceImpl; EventExportServiceImpl	EventFeedback; EventFeedbackInvitation; EventReport; EventReportReminderLog
Contribution/ranking	ContributionBatchController; MemberRankingController	ContributionBatchServiceImpl; ContributionServiceImpl; MemberRankingServiceImpl	ContributionBatch; EventContribution; ContributionAppeal; MemberPerformance; MemberRankingSnapshot
Semester/KPI	AdminSemesterController; SemesterController; ClubDashboardController	SemesterServiceImpl; ClubDashboardServiceImpl; ClubDashboardCalculator	Semester; ClubKPI; ClubEvaluation
Competition	CompetitionController	CompetitionServiceImpl	Competition; CompetitionScore; CompetitionPenalty; CompetitionAward
Posts/notifications	ClubPostController; ClubNotificationController; NotificationController	ClubPostServiceImpl; ClubNotificationServiceImpl; RegistrationNotificationServiceImpl; EventNotificationListener	ClubPost; Notification; NotificationRecipient; EventNotificationDispatch
Knowledge/AI	KnowledgeArchiveController; AIChatController	KnowledgeArchiveServiceImpl; KnowledgeIngestionServiceImpl; AIChatServiceImpl	KnowledgeArchive; KnowledgeChunk; AIChatAuditLog
Configuration/upload/audit	SystemConfigController; UploadController	SystemConfigServiceImpl; UploadServiceImpl; CloudinaryImageStorageService; CloudinaryDocumentStorageService; ClamAvScanService; AuditLogServiceImpl	SystemConfig; AuditLog; SchedulerLog

Appendix E. Deployment Configuration Handoff

Target	Variable	Purpose	Handling rule
Backend / Azure	SEPAY_ACCOUNT_NUMBER	Receiving bank account number used to validate/display payment data	Sensitive operational configuration; do not commit
Backend / Azure	SEPAY_WEBHOOK_API_KEY	Authenticates SePay webhook requests	Secret; backend only; never expose to Vercel
Backend / Azure	PAYMENT_BANK_NAME	Bank display name	Must match frontend
Backend / Azure	PAYMENT_ACCOUNT_NAME	Receiving account holder	Must match frontend
Backend / Azure	PAYMENT_BANK_BRANCH	Bank branch display value	Must match frontend
Frontend / Vercel	VITE_PAYMENT_BANK_ID	Public bank identifier used for QR/display	Public build-time value
Frontend / Vercel	VITE_PAYMENT_BANK_NAME	Public bank display name	Must match Azure bank
Frontend / Vercel	VITE_PAYMENT_ACCOUNT_NUMBER	Public receiving account number	Must equal SEPAY_ACCOUNT_NUMBER
Frontend / Vercel	VITE_PAYMENT_ACCOUNT_NAME	Public account holder name	Must match Azure
Frontend / Vercel	VITE_PAYMENT_BANK_BRANCH	Public branch display value	Must match Azure

Before production handoff, the deployer shall also receive values or ownership instructions for database connectivity, JWT signing, CORS/frontend/backend URLs, Google OAuth2, SMTP, Cloudinary, ClamAV, Gemini, rate limiting/cache if enabled, and logging/monitoring. Secret values should be transferred through an approved password manager or deployment secret store, not chat, source control, screenshots, or the SRS itself.